

Examen RA7: POO

Sistema de gestió d'una llar intel·ligent

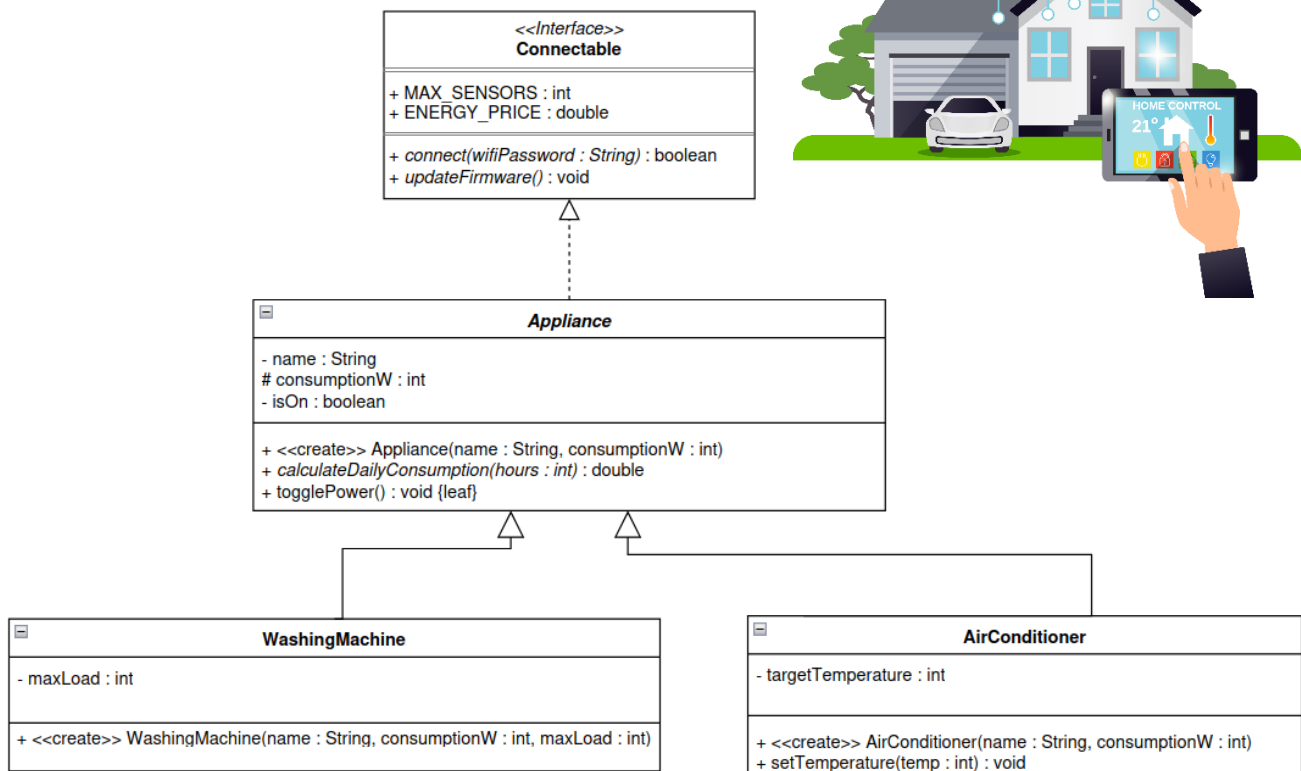
Durada: 2 hores

Context

A l'empresa de domòtica **SmartHome Solutions** on treballes, t'han encarregat el disseny d'un sistema per gestionar el consum i la connectivitat dels electrodomèstics intel·ligents d'una llar. Per aquest motiu, hauràs de programar una estructura de classes que permeti calcular costos energètics i gestionar l'estat dels dispositius.

1. Crea un directori de treball anomenat *ExamenRA7CognomNom*.
2. Crea un package anomenat *smartHome*.

Hauràs de crear i programar les següents classes i interfície, tenint en compte les relacions entre elles que s'expressen al diagrama de classes UML a continuació:



1. Interfície: Connectable (1 pt)

Crea una interfície anomenada `Connectable` que defineixi el comportament de qualsevol dispositiu connectat a la xarxa. Qualsevol element connectat a la xarxa, consumeix energia: l'interfície `Connectable` estableix el preu del KW/h. A més, un dispositiu connectable pot tenir com a màxim 5 sensors (ex: sensor de temperatura, sensor d'humitat, de tancament de porta, etc):

- **Constants:**
 - `MAX_SENSORS = 5`. (int)
 - `ENERGY_PRICE = 0.15` (double)
- **Mètodes:**
 - `boolean connect(String wifiPassword)`
 - `void updateFirmware()`

2. Classe Abstracta: Appliance (4 pts)

Crea la classe abstracta `Appliance` (electrodomèstic) que implementa la interfície `Connectable`. Cada electrodomèstic té un nom, un consum d'energia i un indicador conforme està encès o apagat:

- **Atributs:**
 - `name` (String). Nom de l'electrodomèstic.
 - `consumptionW` (int). Consum diari en Kw/h.
 - `isOn` (boolean). Indicador d'estat (on/off)

A més, cadascun dels electrodomèstics s'ha de poder connectar a la xarxa amb una contrasenya, s'ha de poder actualitzar i s'ha de poder encendre o apagar. La manera de calcular el consum diari és diferent segons el tipus d'electrodomèstic.

- **Constructor:** Ha d'inicialitzar el nom i el consum. Per defecte, `isOn` és `false`.
- **Mètodes a sobreesciure:** (implementar els mètodes de la interfície)
 - `boolean connect(String wifiPassword)` → Retorna 'true' si la contrasenya té més de 5 caràcters. En cas contrari retorna 'false'.
 - `void updateFirmware()` → Mostra el missatge: *"Updating firmware for LG ThinQ..."*
- **Mètode abstracte:**
 - `double calculateDailyConsumption(int hours)`: Donades les hores de funcionament al dia, retorna el cost (en €) del consum diari usant la constant `ENERGY_PRICE`. Serà diferent per cada tipus d'electrodomèstic.
- **Mètode final:**
 - `void togglePower()`: Canvia l'estat d'encès a apagat i viceversa. Mostra el nou estat per pantalla. (Ex: "LG ThinQ is now ON")

3. Subclasses: WashingMachine, AirConditioner

Implementa les classes que hereten d' `Appliance`:

3.1 WashingMachine (2 pts)

- **Atribut propi:** `maxLoad` (int). Càrrega màxima de la rentadora en kg.
- **Mètode a sobreescriure:**
 - `double calculateDailyConsumption(int hours):`
 - **Fórmula:** $(\text{consumptionW} * \text{hours}) * \text{ENERGY_PRICE} / 1000$
 - Si la càrrega màxima és major que 8kg el consum incrementa en un 10%.
 - `toString():` Retorna el nom del dispositiu, el consum i la càrrega màxima.

3.2 AirConditioner (2 pts)

- **Atributs:** `targetTemperature` (int). Temperatura en graus centígrads de l'aire condicionat. En el constructor pren sempre el valor `targetTemperature = 24`.
- **Mètode propi:** `void setTemperature(int temp)`. Estableix la temperatura de l'aire condicionat amb el valor del paràmetre `temp`.
- **Mètodes a sobreescriure:**
 - `double calculateDailyConsumption(int hours):`
 - **Fórmula:** $(\text{consumptionW} * \text{hours}) * \text{ENERGY_PRICE} / 1000$. Si la temperatura de l'aire condicionat és inferior a 22 graus, el consum es duplica.
 - `toString():` Retorna el nom del dispositiu, el consum i la temperatura actual.

4. Classe Main: SmartHomeApp (1 pt)

La classe Main del programa, que ja ve donada i no heu de modificar. L'encertada programació de la resta de classes us portarà a una execució correcta d'aquesta classe. En fer Run, ens mostra per pantalla un missatge com el següent:

```
=== SMART HOME MANAGEMENT SYSTEM ===
Global Energy Price: 0.15 EUR/kWh
-----
Device: LG ThinQ [2000W] (Washing Machine, Max Load: 9kg)
Estimated cost for 5 hours: 1,65 EUR
Updating firmware for LG ThinQ...
LG ThinQ is now ON
-----
Device: Fujitsu Eco [1500W] (Air Conditioner, Target Temp: 20°C)
Estimated cost for 5 hours: 2,25 EUR
Updating firmware for Fujitsu Eco...
Fujitsu Eco is now ON
-----
```

Preguntes Teòriques (1 pt extra)

Contesta les preguntes a) i b) en un comentari a l'inici de la classe *SmartHomeApp.java*. Dóna una resposta breu i concisa:

a) Si dins del mètode `main` intentéssim executar la següent línia de codi:

```
Appliance a = new Appliance("Test Device", 100);
```

Funcionaria la compilació? Justifica la teva resposta basant-te en els conceptes de Programació Orientada a Objectes (POO).

b) Observa el mètode `main` de la classe *SmartHomeApp*. En aquest codi s'aplica el concepte de polimorfisme de dades (o de subtipus).

1. Indica quina línia/es de codi o estructura permet emmagatzemar objectes de diferents tipus en un mateix lloc.
2. Explica breument per què és possible fer-ho i quin benefici aporta al programa.